

Frontend-Testing neu gedacht

Black-Box-Strategie mit Vitest Browser Mode

Alexander Opalic · Otto Payments

TACON 2026

Unser Beispiel: Claw & Chew

Der Test kauft. Der Kunde kann nicht klicken.

```
await user.click(  
  screen.getByRole('button', {  
    name: 'Add The little claw plush to bag',  
  }),  
)  
  
expect(screen.getByLabelText('Bag count'))  
  .toHaveTextContent('1 items in bag')
```



Eine Dekoration liegt über dem Button.

Black-Box-Testing

Ein Kunde legt einen Artikel in den Warenkorb.

Der Warenkorb zeigt diesen Artikel.

- Der Test benutzt die sichtbaren Bedienelemente.
- Die Erwartung beschreibt ein beobachtbares Ergebnis.
- Interne Refs und Methoden bleiben frei veränderbar.

Was soll einen Test brechen?

Änderung	Erwartung
Interne Ref umbenennen	Test bleibt grün
Berechnung in eine Funktion verschieben	Test bleibt grün
Artikel landet nicht im Warenkorb	Test schlägt fehl
Button bekommt einen anderen zugänglichen Namen	Test und Bedienkonzept prüfen
Auch ein Black-Box-Test braucht einen klaren Vertrag.	

Meine Verteilung im ursprünglichen PWA-Projekt

70 %

Integration

20 %

Extrahierte Logik

10 %

Visuell und A11y

Eine Projektentscheidung. Eure Risiken bestimmen eure Verteilung.

Browser Mode im Shop

Teil	Aufgabe
Vitest	Organisiert die Tests und Assertions
Vite	Lädt Testcode, Vue-Komponenten und CSS
Chromium	Führt Test und Komponenten aus
Playwright-Provider	Steuert Browserinteraktionen
Die gerenderte Oberfläche ist Teil des Tests.	

Ein Browser-Projekt

```
browser: {  
  enabled: true,  
  provider: playwright(),  
  instances: [{ browser: 'chromium' }],  
}
```



Vue-Plugin, Stylesheet und Renderer gehören dazu.

Shop: Vitest 5.0.0 · vitest-browser-vue 3.1.0

Derselbe Kauf im Browser

```
const screen = await render(ProductCardHost)

await screen
  .getByRole('button', { name: 'Add The little claw plush to bag' })
  .click({ timeout: 1500 })

await expect.element(screen.getByLabelText('Bag count'))
  .toHaveTextContent('1 items in bag')
```



Der Locator-Klick prüft, ob das Ziel den Klick empfangen kann.

Live: ein blockierter Button

```
pnpm demo blocked-button
pnpm exec vitest run --project jsdom app/catalog/ProductCard.dom.test.ts
pnpm exec vitest run --project browser app/catalog/ProductCard.browser.test.ts
pnpm demo reset
```



jsdom besteht. Der Browser-Klick scheitert.

Danach denselben Browser-Test erneut ausführen.

Eine Bestellung aus Kundensicht

Die Testgrenze liegt um den Shop

Im Test echt

Außerhalb dieser Prüfung

Vue-Komponenten und ihre Events

Nuxt-Server und Hydration

Warenkorb und Mengenänderung

Produktionsrouting

Formular und native Validierung

Zahlungsanbieter

CSS, Fokus und Browserinteraktion

Dauerhafte Bestellung

`render(Shop)` startet die Komponenten direkt.

Die Aktion gehört in den Test

```
const screen = await render(Shop)
await screen.getByRole('button', {
  name: 'Add The little claw plush to bag',
}).click()

await screen.getByRole('button', { name: 'Open bag, 1 items' }).click()
await screen.getByRole('button', {
  name: 'Increase The little claw plush quantity',
}).click()
```



Das Ergebnis gehört ebenfalls in den Test

```
await screen.getByRole('button', { name: 'Checkout' }).click()
await screen.getByRole('textbox', { name: 'Your name' })
  .fill('Claw Fan')
await screen.getByRole('textbox', { name: 'Email address' })
  .fill('fan@example.com')
await screen.getByRole('button', {
  name: 'Place demo order · €60.50',
}).click()

await expect(element(screen.getByRole('heading', {
  name: 'Small claws. Big thank you.',
}))).toBeVisible()
```



Fehlerzustände brauchen eigene Erwartungen

```
await screen.getByRole('button', {  
  name: 'Place demo order · €22.50',  
}).click()  
  
await expect.element(  
  screen.getByRole('textbox', { name: 'Your name' })),  
)toHaveFocus()
```



Ein leeres Formular darf keine Bestellung abschließen.

Pure Logik braucht keine Oberfläche

```
const lines = [aPlushLine({ quantity: 3 })]

expect(summarize(lines)).toEqual({
  subtotal: 8400,
  count: 3,
  shipping: 0,
  total: 8400,
})
```



Drei Plüschtiere erreichen die Grenze für kostenlosen Versand.

Eine Factory macht den Unterschied sichtbar

```
const oneItem = aPlushLine()  
const threeItems = aPlushLine({ quantity: 3 })
```



Der Test verändert genau das relevante Datum.

- Verständliche, feste Standardwerte.
- Pro Aufruf ein neuer Warenkorbbeitrag.
- Keine Zufallsdaten ohne konkreten Testzweck.

Kleine Factory, bekannter Datentyp

```
export function aPlushLine({ quantity = 1 } = {}): CartLine {
  const product = products.find((item) => item.id === 'plush')
  if (!product) {
    throw new Error('The talk fixture requires the plush product')
  }
  return {
    key: 'plush-fixture',
    product,
    variant: 'One size · 18 cm',
    print: null,
    quantity,
  }
}
```



Ein Page Object bündelt wiederholte Bedienung

```
const shop = await renderShop()

await shop.addPlushAndCheckout()
await shop.enterCustomer({
  name: 'Claw Fan',
  email: 'fan@example.com',
})
await shop.placeOrder()

await expect.element(shop.confirmation).toBeVisible()
```



Die Interaktion bleibt nachvollziehbar

```
async enterCustomer({ name, email }: {  
  name: string  
  email: string  
}) {  
  await screen.getByRole('textbox', { name: 'Your name' }).fill(name)  
  await screen.getByRole('textbox', { name: 'Email address' }).fill(email)  
}
```



Ein Helfer für Wiederholung. Die fachliche Erwartung bleibt im Test.

Rollenbasierte Suche braucht verständliche Namen

```
screen.getByRole('textbox', { name: 'Email address' })  
screen.getByRole('button', { name: 'Checkout' })
```



Fehlt der zugängliche Name, fehlt dem Test sein Ziel.

Das prüft einen Teil der Semantik. Es ersetzt kein Accessibility-Audit.

Nach Escape kehrt der Fokus zurück

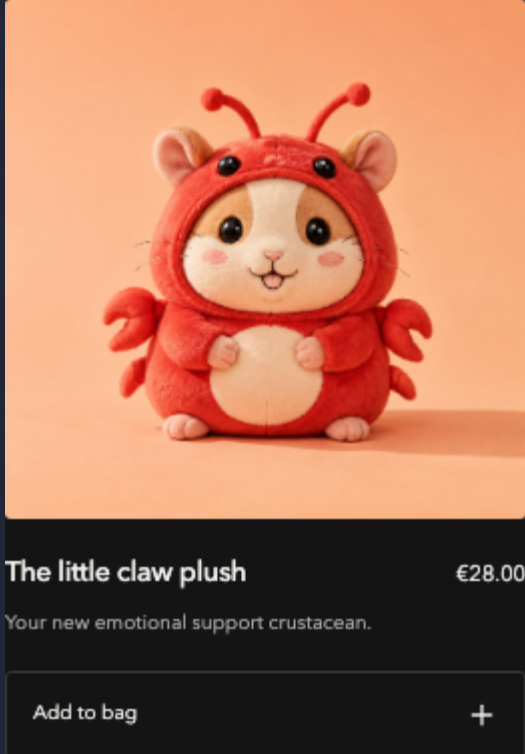
Keine Verstöße ist nicht immer eine vollständige Prüfung

```
const results = await axe.run(notice.element(), {  
  runOnly: ['color-contrast'],  
})  
  
expect(results.violations.map((rule) => rule.id)).toEqual([])  
expect(results.incomplete).toEqual([])
```



Der Checkout-Test prüft zusätzlich, dass die Kontrastregel ausgeführt wurde.

Ein Screenshot schützt die Produktkarte



Ein definierter Zustand.

Der Screenshot-Vergleich

```
await artwork.decode()  
await document.fonts.ready  
  
await expect(  
  screen.getByRole('article'),  
)toMatchScreenshot('plush-card')
```



Gleicher Browser, gleiche Umgebung, feste Bildgröße.
Referenzen prüfen. Änderungen bewusst übernehmen.

Ein visueller Fehler bleibt bedienbar

Referenz



The little claw plush

€28.00

Your new emotional support crustacean.

Add to bag



Geänderte Darstellung

Die gebaute Anwendung hat zusätzliche Risiken

Test lädt

Kann hier belegen

`render(Shop)`

Komponenten funktionieren zusammen

Die gebaute Nuxt-Anwendung

Server-HTML und Client passen zusammen

Produktionsrouten

Produktbilder erreichen den Browser

Im Hydration-Demo besteht Browser Mode.

Der Playwright-Anwendungstest erkennt den Fehler.

Der erste Schritt in eurem Projekt

1. Einen wichtigen Nutzerablauf auswählen.
2. Eine passende Testgrenze festlegen.
3. Über sichtbare Bedienelemente testen.
4. Einen echten Fehler einbauen und den Test scheitern sehen.
5. Wiederholungen erst dann in Helfer verschieben.

Was beweist euer grüner Test?

Der Kunde kann handeln.

Das erwartete Ergebnis tritt ein.

Fragen?

Alexander Opalic · alexop.dev

Backup

Die vierfache Geschwindigkeit im Abstract

Gemessene Suite im ursprünglichen Artikel	Laufzeit
Browser Mode	13,59 s
JSDOM	53,72 s

82 Tests. Beide Läufe hatten vier Fehler.

Ein Projektvergleich, kein allgemeiner Geschwindigkeitsfaktor.

Weitere vorbereitete Experimente

Beispiel	Beobachtung
Unscrollbarer Warenkorb	Layout und echte Wheel-Eingabe
T-Shirt nach Resize verschieben	Geometrie und Pointer-Eingabe
Größe L auswählen	Erwarteter Zustand im ARIA-Snapshot
Fehlerhafte Bildroute	Unterschied zwischen Vite und Nuxt-Auslieferung
Die interaktive Erklärung liegt im Shop unter <code>/learn</code> .	

Beispiele ausführen

```
# Im Verzeichnis claw-and-chew
pnpm test:watch
pnpm exec vitest run --config talk/vitest.tacon.config.ts --project unit
pnpm exec vitest run --config talk/vitest.tacon.config.ts --project browser
pnpm exec vitest run --config talk/vitest.tacon.config.ts --project visual
```



Die Screenshot-Referenz gehört zur verwendeten Umgebung.